Copy Propagation, Constant Propagation and Constant Folding

Works on straight-line three-address code. Each statement has the form x = y or x = a op b, with op one of + - * / and integer constants.

Program (opt.c)

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <stdlib.h>

#define MAX 20

typedef struct { char res[10], a[10], b[10], op; } Stmt;   /* res = a op b  (op = 0 -> res = a) */

Stmt orig[MAX], code[MAX];
int n;
char var[MAX][10], val[MAX][10];                           /* propagation table */
int cnt;

int isnum(char *s) {
    if (!*s) return 0;
    for (; *s; s++) if (!isdigit(*s)) return 0;
    return 1;
}

int find(char *name) {
    for (int i = 0; i < cnt; i++)
        if (strcmp(var[i], name) == 0) return i;
    return -1;
}

void removeAt(int i) {
    for (; i < cnt - 1; i++) { strcpy(var[i], var[i+1]); strcpy(val[i], val[i+1]); }
    cnt--;
}

void kill(char *name) {                  /* remove entries that use 'name' as key or value */
    for (int i = 0; i < cnt; ) {
        if (strcmp(var[i], name) == 0 || strcmp(val[i], name) == 0) removeAt(i);
        else i++;
    }
}

void replaceUses(Stmt *s) {              /* substitute operands using the table */
    int i = find(s->a);
    if (i >= 0) strcpy(s->a, val[i]);
    if (s->op && (i = find(s->b)) >= 0) strcpy(s->b, val[i]);
}

void copyPropagation() {
    cnt = 0;
    for (int i = 0; i < n; i++) {
        replaceUses(&code[i]);
        kill(code[i].res);
        if (!code[i].op && !isnum(code[i].a) && strcmp(code[i].res, code[i].a) != 0) {
            strcpy(var[cnt], code[i].res);
            strcpy(val[cnt], code[i].a);
            cnt++;
        }
```

```
    }
}

void constantPropagation() {
    cnt = 0;
    for (int i = 0; i < n; i++) {
        replaceUses(&code[i]);
        kill(code[i].res);
        if (!code[i].op && isnum(code[i].a)) {
            strcpy(var[cnt], code[i].res);
            strcpy(val[cnt], code[i].a);
            cnt++;
        }
    }
}

void constantFolding() {
    for (int i = 0; i < n; i++) {
        if (code[i].op && isnum(code[i].a) && isnum(code[i].b)) {
            int x = atoi(code[i].a), y = atoi(code[i].b), r = 0, ok = 1;
            switch (code[i].op) {
                case '+': r = x + y; break;
                case '-': r = x - y; break;
                case '*': r = x * y; break;
                case '/': if (y) r = x / y; else ok = 0; break;
                default: ok = 0;
            }
            if (ok) { sprintf(code[i].a, "%d", r); code[i].op = 0; }
        }
    }
}

void show() {
    for (int i = 0; i < n; i++) {
        if (code[i].op) printf("  %s = %s %c %s\n", code[i].res, code[i].a, code[i].op, code[i].b);
        else            printf("  %s = %s\n", code[i].res, code[i].a);
    }
}

int main() {
    char line[100], c;
    int i, r, ch;

    printf("Number of statements: ");
    fgets(line, sizeof(line), stdin);
    sscanf(line, "%d", &n);

    printf("Enter statements (like  a = 5   b = a   c = b + 3):\n");
    for (i = 0; i < n; i++) {
        fgets(line, sizeof(line), stdin);
        r = sscanf(line, "%9s = %9s %c %9s", orig[i].res, orig[i].a, &c, orig[i].b);
        orig[i].op = (r == 4) ? c : 0;
    }

    do {
        printf("\n1. Copy Propagation\n2. Constant Propagation\n3. Constant Folding\n"
               "4. All three (in this order)\n5. Exit\nChoice: ");
        fgets(line, sizeof(line), stdin);
        sscanf(line, "%d", &ch);
        if (ch < 1 || ch > 4) continue;

        memcpy(code, orig, sizeof(orig));
        if (ch == 1 || ch == 4) copyPropagation();
        if (ch == 2 || ch == 4) constantPropagation();
        if (ch == 3 || ch == 4) constantFolding();
```

```
        printf("\nOptimized code:\n");
        show();
    } while (ch != 5);

    return 0;
}
```

How to run

```
gcc opt.c -o opt
./opt
```

Sample input

```
5
a = 5
b = a
c = b + 3
d = 2 * 4
e = c + a
```

Output for each choice

```
1. Copy Propagation      2. Constant Propagation   3. Constant Folding   4. All three
   a = 5                    a = 5                     a = 5                  a = 5
   b = a                    b = 5                     b = a                  b = 5
   c = a + 3                c = 5 + 3                 c = b + 3              c = 8
   d = 2 * 4                d = 2 * 4                 d = 8                  d = 8
   e = c + a                e = c + 5                 e = c + a              e = c + 5
```